## Dinamik Veri Yapısı

* Dinamik veri yapısı, programın çalışma zamanında ihtiyaca doğrultusunda büyüyebilen ve küçülebilen veri yapısıdır.

## Static Veri Yapısı

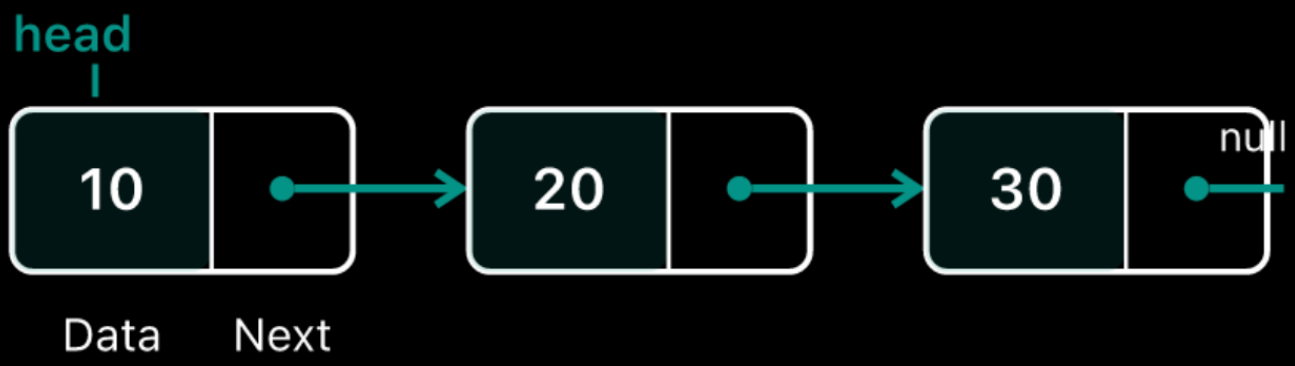
* Statik veri yapısı, boyutu genellikle oluşturulurken belirlenen çalışma sırasında doğrudan küçülüp büyüyemeyen veri yapısıdır.

| Özellik | Static | Dinamik |
|---|---|---|
| Boyut | Genellikle sabit | Değişebilir. |
| Boyut Belirleme | Önceden | Çalışma zamanında |
| Bellek | Önceden ayrılır | İhtiyaca göre |
| Esneklik | Düşük | Yüksek |
| Yönetim | Kolay | Karmaşık |
| Ek bellek maliyeti | Düşük | Bağlantılar vb. olabilir |
| Örnek | Array | Linked list |
| Bellek yönetimi | Basit | Dikkat gerekir. |

## II. Bölüm - Dinamik Veri Yapıları

### Linked List

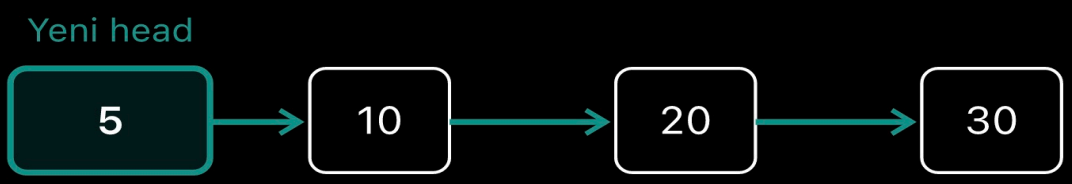
* Bağlı liste, düğümlerin birbirlerine referans oracılığıyla bağlanmasıyla oluşan dinamik ve doğrusal bir veri yapısıdır.
* Her iki düğüm iki temel olan içerir:
  - Data: Düğümün içinde saklanan veri
  - Next: Bir sonraki düğümün referansı




Tek yönlü bağlı liste: 10 → 20 → 30 → null

* Bağlı listenin özellikleri
  - Dinamik boyuttadır.
  - Düğümler bellekte ardışık olmak zorunda değildir.
  - Elemanlar sırayla eşlenir.
  - Düğüm ekleme ve silme bağlantılar değiştirilerek yapılır.
  - Her düğüm veri ve referans bilgisi saklar.
  - İlk düğüm head referansı bulunur.
* Bağlı listede ekleme

### A. Başa eleman ekleme

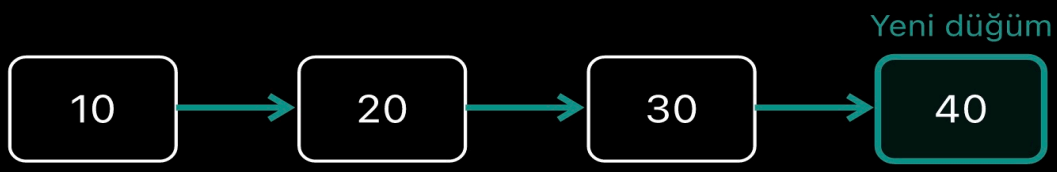



Yeni düğüm ilk düğümü gösterir, ardından `head` yeni düğüme atanır.

```java
yeni.next = head;
head = yeni;
```

**Zaman karmaşıklığı: O(1)**

### B. Sona eleman ekleme

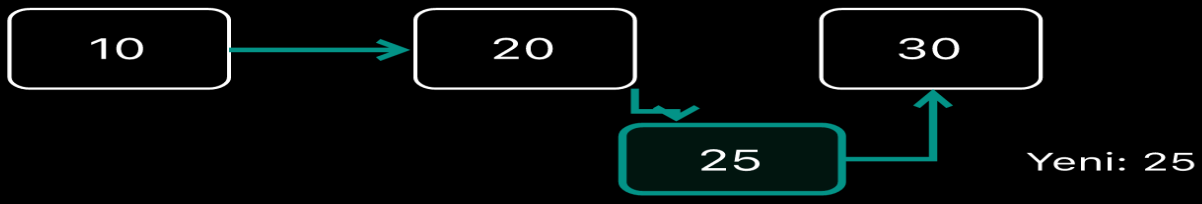



Son düğüme kadar ilerlenir ve son düğümün `next` alanına yeni düğüm bağlanır.

```java
Node gecici = head;
while (gecici.next != null) {
    gecici = gecici.next;
}
gecici.next = yeni;
```

**Zaman karmaşıklığı: O(n)** (son düğüm referansı tutulmuyorsa).

### C. Araya eleman ekleme




Önce yeni düğüm sonraki düğüme, ardından önceki düğüm yeni düğüme bağlanır.

```java
yeni.next = gecici.next;
gecici.next = yeni;
```

**Zaman karmaşıklığı: O(1)** (ekleme yapılacak önceki düğüm biliniyorsa).